Data Handling in R

2026-07-15

Abstract Corrections of and comments on the syllabus text, the example scripts, and answers to selected assignments are solicited. Requests for additions are welcome as well. Future students will appreciate your input! We acknowledge valuable contributions by former teachers Ozan Aksoy, Jos Dessens, Wim Jansen, Peter van de Heijden, Peter Ligtig, David Macro, Vardan Barsegyan, and Weverthon Machado as well as by numerous students.

Table of contents

1 Introduction

- Contents via shortcode (see below)

2 Getting to know and summarizing data

In this part you will learn about the following basic tasks:

- Change your working directory/folder.
- Load a dataset from disk into computer memory.
- Explore the variables in the data.
- List values for (some) variables.
- Sort the observations on variables.
- The concept of "key variable(s)".
- Create frequency distributions of variables.

- Create cross tabulations of multiple variables.
- Produce a table of summary statistics (e.g., means, sd = standard deviation) for a series of variables.
- Produce a table of correlations of quantitative variables.

The concept of key variables may be new to you. A key consists of one or more variables that uniquely identify observations.

In social science datasets, we often use single observations to represent all variables measured on subjects. Each person is then a row in the data matrix. Such datasets often contain a respondent number, say `id`, usually integer valued. The values of `id` are usually unique, and then `id` qualifies as a key. If you know a subject's value of `id`, you can find the corresponding row in the data matrix, and hence know all the subject's characteristics.

If a value of `id` occurs more than once, obviously, the value of `id` does not identify a unique subject, and `id` is not a key. Observe that `id` need not take all possible values. For instance, if your `id` consists of 9 digits, likely your dataset does not contain all possible numbers of 9 digits. Many possible values of `id` will not be used. That is not a problem. `id` would still be a key if all values are distinct.

If `id` takes duplicate values, we may form a key by considering additional variables. For instance, think of a household survey with one or two adults. Observations are people nested in households. The variable `hid`, household number, would not be a key. The combination of `hid` and `sex` might be a key if the sample comprises single people and different-sex couples, but not if the sample also includes same-sex couples, cohabiting siblings, etc. In the latter case, the data may contain a variable indicating whether a person is the oldest adult in the household, and together with `hid` this variable may form a key. If you only have birth dates, the combination of `hid` and birth date is usually a key, but you could have bad luck if your data include a household with two cohabiting twin sisters.

If a dataset does not contain a case identifier such as a respondent number, we strongly suggest you create one. Be sure to do this in a reproducible way. First put the observations in a specific order, by sorting on a series of variables that uniquely identify cases. In the example below, we assume that each observation has a unique pattern on the variables `city`, `age`, `edu`, and `sex`. Obviously, you should verify this assumption.

In R, you can check duplicates and create an identifier as follows:

```
df %>%
  count(city, age, edu, sex) |>
  filter(n > 1)

df <- df |>
  arrange(city, age, edu, sex) |>
  mutate(respnr = row_number())
```

After sorting the cases, a new variable is created with the value 1 in the first observation, 2 in the second observation, 3 in the third observation, etc. This is one of the occasions in which you may want to save the dataset, but we urge you not to overwrite the original file.

2.1 R

You will learn the R functions listed below to describe and summarize data. Read the corresponding documentation pages when needed. Start with the description and examples. We do not expect you to learn all functions by heart, but you should get a rough idea, and remember, what they can accomplish.

Function	Description
`read_csv()`	Load a CSV file into R
`read_dta()`	Load a Stata file into R
`read_excel()`	Load an Excel file into R
`glimpse()`	Describe data contents
`skim()`	Describe data contents and summary statistics
`View()`	View observations and variables
`arrange()`	Sort observations
`table()`	One-way and two-way tables of frequencies
`prop.table()`	Convert frequencies to proportions
`tabyl()`	Frequency tables with percentages
`chisq.test()`	Pearson's chi-square test for independence
`summarise()`	Summary statistics
`cor()`	Pearson correlations
`count()`	Count observations by category or group
`filter()`	Select observations satisfying a condition

Examples:

Load a file into memory, using an R Project:

```
library(readr) # for opening CSV files
data <- read_csv("data/mydata.csv")

library(haven) # for opening Stata (.dta) files
data <- read_dta("data/mydata.dta")
```

```
library(readxl) # for opening Excel (.xlsx) files
data <- read_excel("data/mydata.xlsx")
```

Use relative file paths, not full paths to your computer.

Describe variables in the active dataset:

```
glimpse(data) # describe all variables

data |>
  select(edu, exp) |>
  glimpse() # compact overview of edu and exp

data |>
  select(starts_with("rat")) |>
  glimpse() # describe all variables starting with rat

skim(data) # extended summary statistics
View(data) # open data viewer in RStudio
```

List observations:

```
data # all variables and all observations

data |>
  select(edu, inc) # selection of variables

data |>
  filter(country == 4) # selection of observations

data |>
  select(country, edu, inc) |>
  slice(1:20) # selection of variables and observations 1 to 20
```

Sort observations on one or more variables in increasing order:

```
data |>   arrange(edu) # sort by edu

data |>   arrange(edu, inc) # sort by edu and inc
```

One-way tabulation, frequencies:

```r
table(data$class) # one-way table

table(data$edu, useNA = "ifany") # one-way table, include missing values

data |>
  tabyl(edu) # frequency table with percentages
```

Two-way tabulation of variables:

```r
tab <- table(data$edu, data$class)
tab
# two-way table
```

Row and column percentages:

```r
prop.table(tab, margin = 1)
# row percentages

prop.table(tab, margin = 2)
# column percentages
```

The argument `margin = 1` computes proportions within rows. The argument `margin = 2` computes proportions within columns.

Pearson's chi-square test for independence:

```r
chisq.test(tab)
# Pearson's chi-square test for independence
```

The chi-square test checks whether two categorical variables are statistically independent. In this example, it tests whether edu and class are associated.

Tabulation of three or more variables:

```r
data |>
  group_by(gender) |>
  count(edu, class) # two-way table separately by gender

data |>
  group_by(country, gender) |>
  count(edu, class) # two-way table separately by country and gender

xtabs(~ edu + class + gender, data = data) # table of three variables
```

Summary statistics:

```
data |>
  summarise(
    mean_income = mean(income, na.rm = TRUE),
    sd_income = sd(income, na.rm = TRUE),
    min_income = min(income, na.rm = TRUE),
    max_income = max(income, na.rm = TRUE)
    ) # summary statistics for income
```

Correlation matrix of two or more variables:

```
data |>
  select(edu, inc, age) |>
  cor(use = "complete.obs") # Pearson correlations
```

Count the number of observations meeting a condition:

```
data |>
  filter(income > 50000) |>
  nrow() # count observations with income above 50000

data |>
  filter(income > 5000, income < 50000) |>
   nrow() # count observations with income between 5000 and 50000, boundaries
excluded

data |>
  filter(between(income, 5000, 50000)) |>
   nrow() # count observations with income between 5000 and 50000, boundaries
included

data |>
  filter(income > 50000, !is.na(income)) |>
  nrow() # count observations with income above 50000 and income not missing

data |>
  filter(income > 50000, sex == 1) |>
  nrow() # count observations with income above 50000 and sex equal to 1
```

Check that the variable `birthd` is a key:

```
data |>
  summarise(no_missing = all(!is.na(birthd))) # birthd is never missing

data |>
  count(birthd) |>
  filter(n > 1) # check duplicates in birthd
```

Check that `birthd` and `sex` form a composite key:

```
data |>
  summarise(no_missing = all(!is.na(birthd), !is.na(sex))) # birthd and sex are
never missing

data |>
  count(birthd, sex) |>
  filter(n > 1) # check duplicates in the combination of birthd and sex
```

Create a key:

```
data <- data |>
  arrange(birthd, sex) |>
   mutate(id = row_number()) # create id after sorting by birthd and sex Data
Visualization with ggplot2
```

2.2 Assignments

In these assignments we use the dataset `sbs399.dta`. Solutions can be found in the file `dh_summary_assignments.Rmd`. Be sure to start your R script in the style of the template files.

1. What is the variable label of `feduc`?
2. Create an overview of the variables `feduc` and `meduc`.
3. Display a frequency distribution of `educ`. What is the percentage of subjects with an education up to category 4? How many missing values are there in `educ`?
4. List the values of the variables `hrinc` and `pres`. Adjust the syntax to list only the males among the initial 40 observations. Further adjust the syntax to restrict the sample to males with below average education among the initial 40 observations. Finally, adjust the syntax to list males and females separately among the initial 5 observations.
5. List values of `age` in increasing order. Next, list just the five smallest values, and list just the five largest values. Do not read these values from the data viewer. Give syntax to display the requested cases. Then list all observations except the observations between 6 and 3345, both numbers included.

6. Create a two-way table of `feduc` rows and `meduc` columns. Add row and column percentages. Test for independence of `feduc` and `meduc` using Pearson's chi-square test.
7. What are the mean, standard deviation, minimum, and maximum value of `feduc`? And what is the median?
8. Verify that the number of subjects for whom the mother is higher educated than the father is 543. What is the count if you restrict to female subjects?
9. What are the Pearson correlations of `educ`, `feduc`, and `meduc`? Use the help system to find out how to compute Spearman's rank correlations.
10. Is it true that for any variable the values are above average for 50% of cases? If you are uncertain, check this with some variables. For an explanation, think about this example: what percentage of people have more than the average number of legs? 50%? Who are these people?
11. Is `hrespnr` a key, meaning that it is never missing and all values are unique? Why? Do not rely on the data viewer. Write syntax to obtain the information on which you base your answer. What about the pair of variables `hrespnr` and `sex`? Why? What about `hrespnr`, `sex`, and `age`, and `hrespnr`, `sex`, and `fage`?

3 Documenting data

In this short section you will learn about the following basic tasks:

- Rename variables and give them clear names.
- Use meaningful names for categorical values.
- Convert categorical variables to factors.
- Code missing values as `NA`.
- Change the order of variables in a dataset.

3.1 R

Learn the following R commands to document the data, variables, and values:

Function	Description
`rename()`	Rename variables
`mutate()`	Modify or create variables
`factor()`	Convert a variable to a factor
`na_if()`	Change a specific value to `NA`
`replace_na()`	Replace `NA` with a specific value
`relocate()`	Change the order of variables

Rename variables:

```
data <- data |>
  rename(educ = v101)
# rename v101 to educ

data <- data |>
  rename(sex = v1, edu = v2)
# rename multiple variables
```

Convert a categorical variable to a factor:

```
data <- data |>
  mutate(
    sex = factor(
      sex,
      levels = c(1, 2),
      labels = c("male", "female")
    )
  )
# assign meaningful labels to categories
```

Code missing values:

```
data <- data |>
  mutate(educ = na_if(educ, 99))
# change the value 99 to NA

data <- data |> mutate(across(c(educ, feduc), ~ na_if(.x, 9)))
# change the value 9 to NA in educ and feduc
```

Replace missing values with a numeric code:

```
data <- data |>
  mutate(educ = replace_na(educ, 99))
# change NA to 99
```

Change the order of variables:

```
data <- data |>
  relocate(hrespnr, sex, age, edu)
# put these variables first
```

Be careful when replacing missing values with numeric codes such as 99. In R, missing values should usually be coded as `NA`. Numeric missing-value codes can easily be treated as real values in analyses if they are not recoded properly.

3.2 Assignments

See `dh_doc.Rmd` for solutions.

In these assignments we use the dataset `sbs399_unlabeled.dta`.

1. Rename the variable `hrespnr` to `hid` and the variable `hrinc` to `hinc` in one command.
2. The variable `sex` is coded numerically. Convert `sex` into a factor with meaningful labels. Use `male` and `female` as category labels.
3. The variables `sexrol1` and `sexrol2` measure family attitudes. Convert both variables into factors with the following category labels:
    1. fully disagree
    2. slightly disagree
    3. indifferent
    4. slightly agree
    5. fully agree